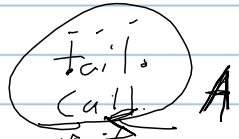
writing no record test #1.

Handling Tail calls

method
...
ret

method A



method B
① method exit?
└ ② handle some other way?
tail
call A

end method

Event log

method	A	enter	TS
m	B	enter	TS
"	B	exit	TS
	A	exit	
method	B	enter	
method	A	enter	TS1
	A	enter	TS2
	"	enter	TS3
	A	exit	
	A	exit (optional)	
	B	exit	

method A (depth)
if depth > 1
tail call A
else
ret

- to figure out if method A is being tail called from the events log, do

		timestamp
B	enter	1
A	enter	2
A	enter	3
A	exit	4 (optional)
B	exit	

pattern to look for is more 'enter' than 'exit' within the context of 'B'

method A

method A

method A

example → - towers of hanoi

- merge sort

- fibonacci - ~~recursive~~ naive impl is not tail callable

not tail callable

~~method B~~

~~tail method A~~

~~Exit~~

method B

call helper

method helper

call B

method setup

call recurse

method recurse

tail recurse

returns

setup enter

recurse enter

" "

" "

. . . .

recurse exit (optional)

setup exit

```
method A
  if
    A tail event
    tail A
  else if
    call B
  else
    exit
method B
  if
    tail B
  else if
    call A
  else
    exit
```

```
A   enter
 :    :
 :    :
B   enter
B   exit
A   enter
 :    :
 :    :
B   enter
 :    :
 :    :
```

assume that A tail calls itself.

```
B exit
A exit
```
option 1

```
A enter
A exit
```
option 2

```
A enter
A  :
A  :
A exit
```
option 3

no instr here

tail call ← no code
ret here

```
A enter
A tail
A enter
```

resolution:
- tail call event is nice and helps, but isn't required to interpret the event log correctly. accurately.

Won't do for now. will write tests